



State Refactoring

Reorganizing how Terraform tracks your infrastructure without changing the infrastructure itself

Common scenarios:

- ✧ You renamed a resource in your code
- ✧ You're reorganizing modules
- ✧ You need to adopt existing infrastructure
- ✧ You want to stop managing something but keep it running

Without refactoring tools, Terraform thinks you're destroying old resources and creating new ones





The Old Way

vs

The New Way

- ❯ Used separate CLI commands (terraform state mv, terraform state rm, terraform import)
- ❯ Commands are run outside your configuration
- ❯ Not version-controlled
- ❯ Hard to review in pull requests
- ❯ Risky - one wrong command and you've modified state permanently

- ❯ Use configuration blocks (moved, removed, import)
- ❯ Declared in your `.tf` files
- ❯ Version-controlled alongside your infrastructure
- ❯ Reviewable in PRs
- ❯ Part of the standard `plan/apply` workflow
- ❯ Can be tested before being applied



Core Refactoring Blocks

moved block

```
1 moved {
2   from = azurerm_subnet.subnet1
3   to   = azurerm_subnet.prod_private
4 }
```

Purpose

Rename or relocate resources in state

When to Use

Refactoring code structure, renaming resources

removed block

```
1 removed {
2   from = aws_instance.production_db
3
4   lifecycle {
5     destroy = true
6   }
7 }
```

Purpose

Stop managing a resource but keep it running

When to Use

Handing off ownership, removing from Terraform control

import block

```
1 import {
2   to = aws_s3_bucket.terraform_state
3   id = "terraform_state_bucket"
4 }
```

Purpose

Adopt existing infrastructure into Terraform

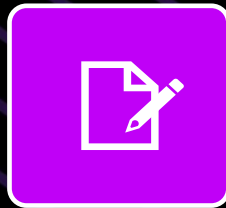
When to Use

Bringing manually-created resources under management



The Refactoring Workflow

Write



Add blocks to
Terraform config

Plan



terraform
plan
(validate changes)

Apply



terraform
apply
(updates state)

LECTURE

Moved Block



The Real-World Scenario...



- ❧ After building our infrastructure, we realize that our naming isn't clear...
- ❧ Blocks are very similar, but the names aren't descriptive in relation to our resources...
- ❧ We need to refactor, but what happens when we rename this?

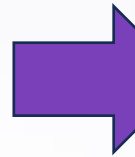
```
resource "aws_vpc" "vpc" {  
  cidr_block = var.sandbox_vpc_cidr  
  tags = {  
    Name      = var.sbx_vpc_name  
    Environment = "sandbox_network"  
    Terraform  = "true"  
  }  
}  
  
resource "aws_vpc" "vpc1" {  
  cidr_block = var.test_vpc_cidr  
  tags = {  
    Name      = var.test_vpc  
    Environment = "testing_network"  
    Terraform  = "true"  
  }  
}
```

You Decide to Rename Using a Naming Standard...



```
main.tf

resource "aws_vpc" "vpc" {
  cidr_block = var.sandbox_vpc_cidr
  tags = {
    Name      = var.sbx_vpc_name
    Environment = "sandbox_network"
    Terraform  = "true"
  }
}
```



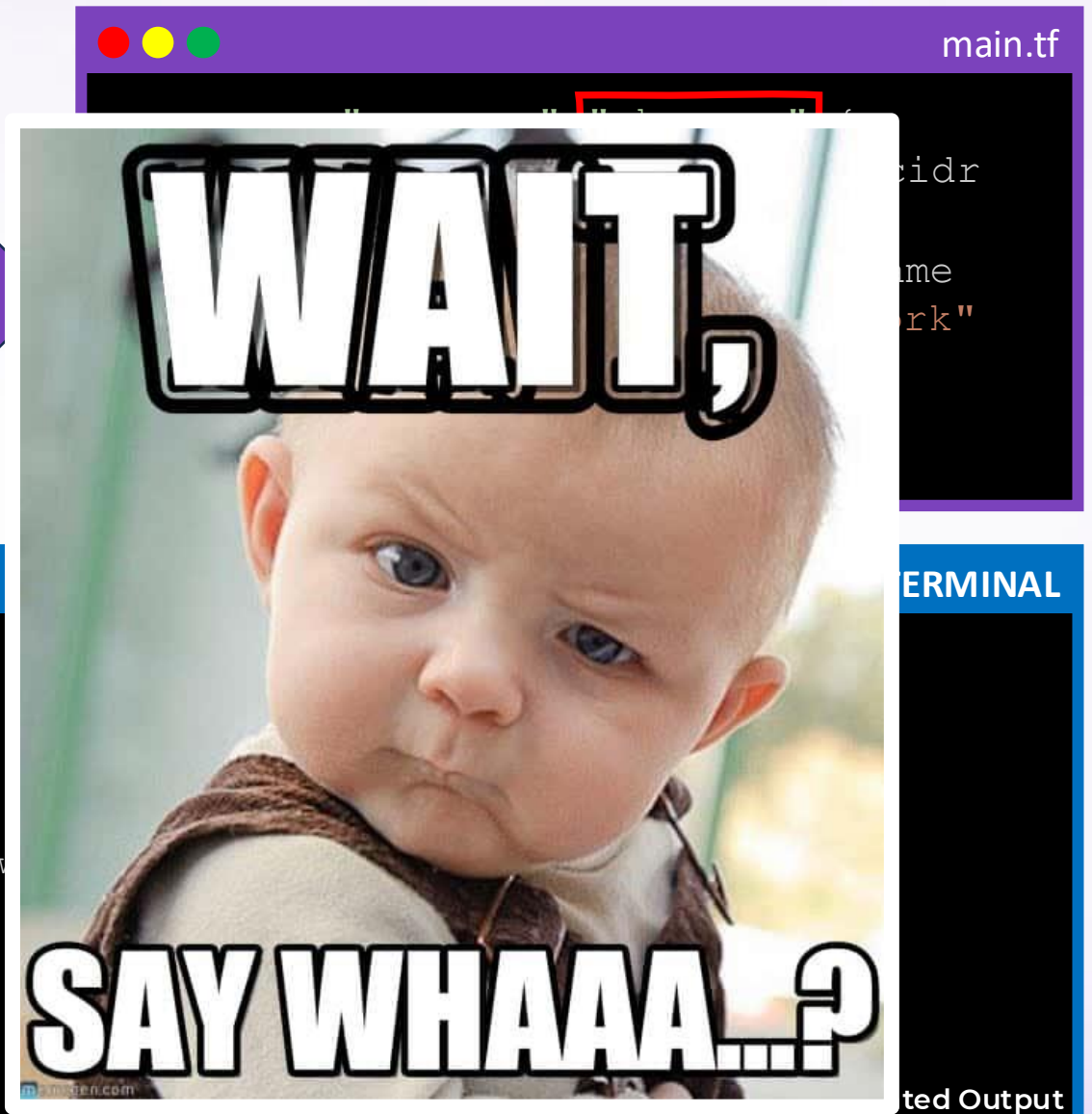
```

$ terraform plan
Terraform will perform the following actions:

# aws_vpc.prn_vpc will be created
+ resource "aws_vpc" "prd_vpc" {
  + arn                                = (known after apply)
}

# aws_vpc.vpc will be destroyed
# (because aws_vpc.vpc is not in configuration)
- resource "aws_vpc" "vpc" {
}

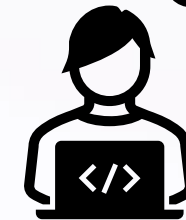
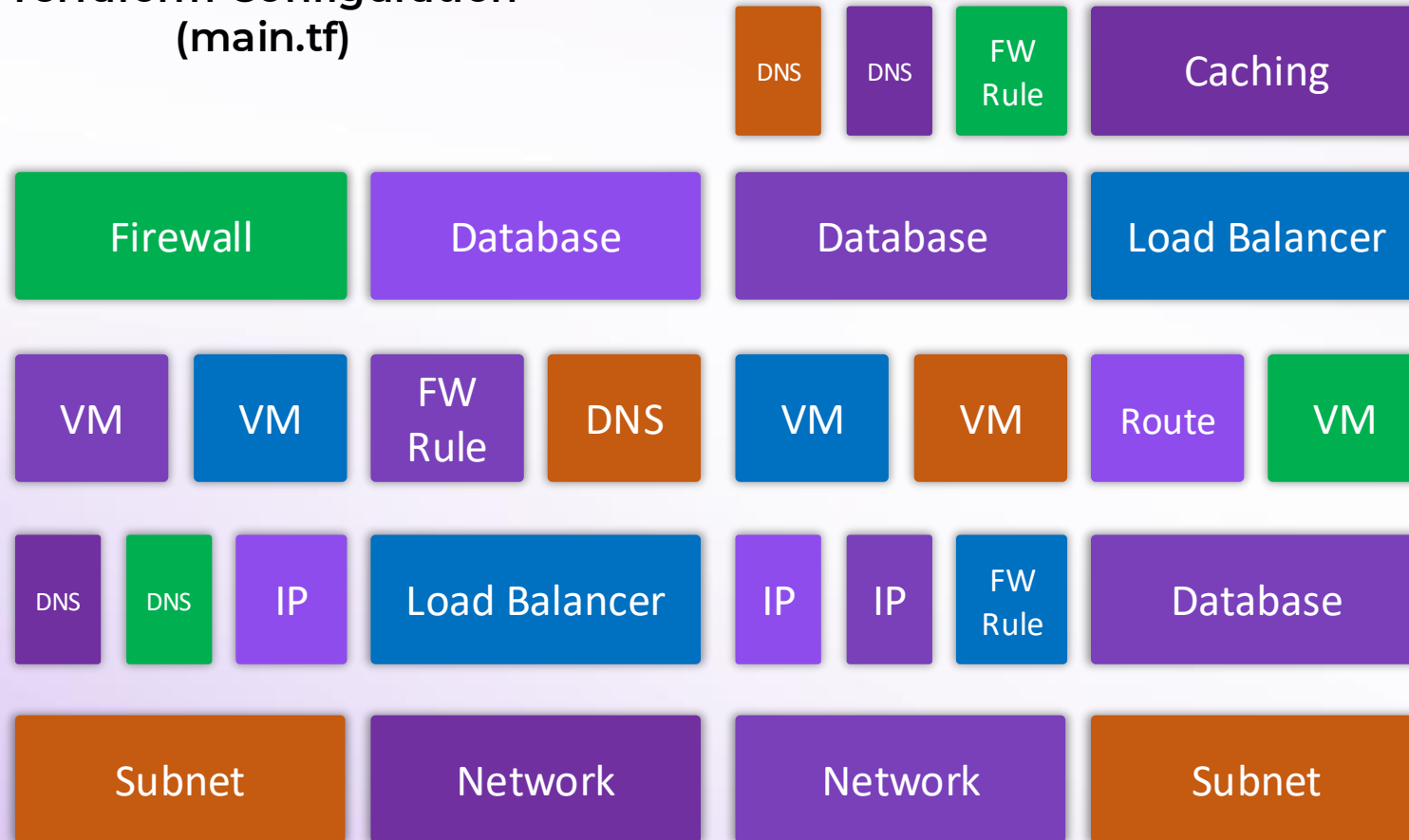
Plan: 1 to add, 0 to change, 1 to destroy.
```



Another Scenario...

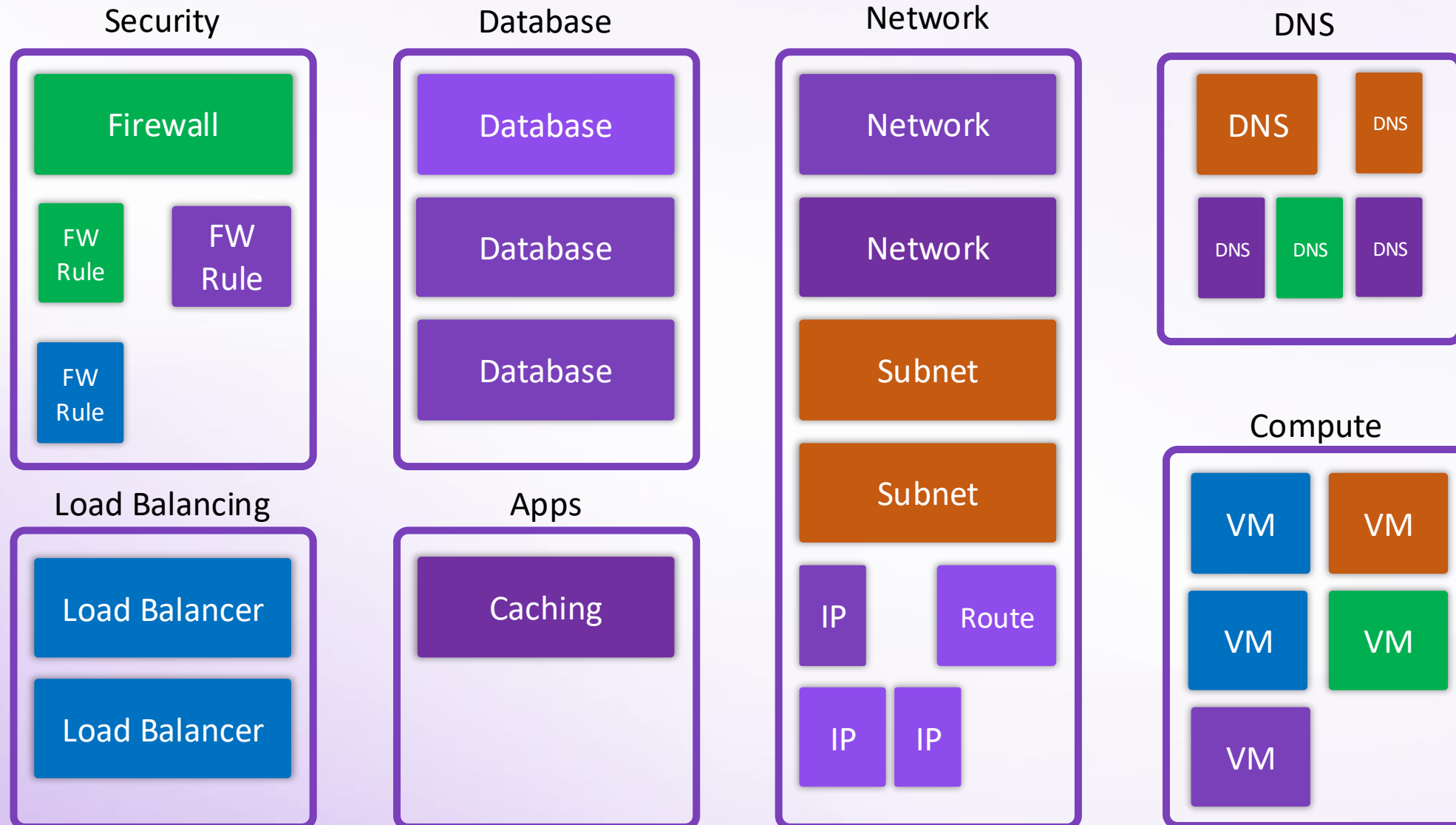


Terraform Configuration
(main.tf)



"I need to
reorganize this"

Another Scenario...





Moved Block

The `moved` block tells Terraform you've renamed or relocated a resource in your code. It updates the state file to reflect the new address without destroying and recreating your actual infrastructure.

Using the `moved` block prevents downtime and data loss during refactoring. It's your safety net when cleaning up resource names or restructuring your configuration.



Moved Block

The moved block updates state to reflect refactored resource addresses without destroying infrastructure.



Two Parameters

`from` = specify the original resource address
`to` = specify the new resource address



State Updates Only

Updates only the state file - no infrastructure changes are made



Resource Blocks Must Exist

You MUST write the new `resource` blocks before refactoring resource address



```
moved {  
  from = aws_instance.server  
  to   = aws_instance.web_server  
}  
  
moved {  
  from = aws_s3_bucket.data  
  to   = module.storage.aws_s3_bucket.data  
}  
  
moved {  
  from = module.old_app  
  to   = module.new_app  
}
```

Breaking Down the Moved Block



```
moved {  
  from = aws_instance.server  
  to   = aws_instance.web_server  
}  
  
moved {  
  from = aws_s3_bucket.data  
  to   = module.storage.aws_s3_bucket.data  
}
```

Where was the original resource block managing this infrastructure?

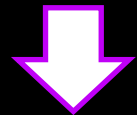
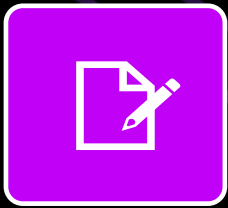
Where is the NEW resource block that will manage this infrastructure?

An example showing that you want to move the resource block into a new module.



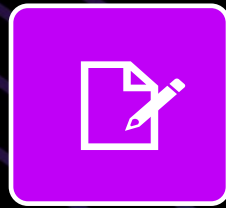
The Refactoring Workflow

Write



**Add new
resource blocks
to Terraform
config**

Write



**Add new moved
blocks to
Terraform config**

Plan



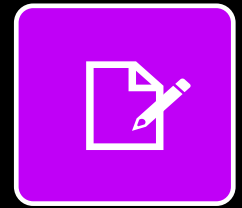
**terraform
plan
(validate)**

Apply



**terraform
apply
(updates state)**

Delete



**Delete the
moved blocks**

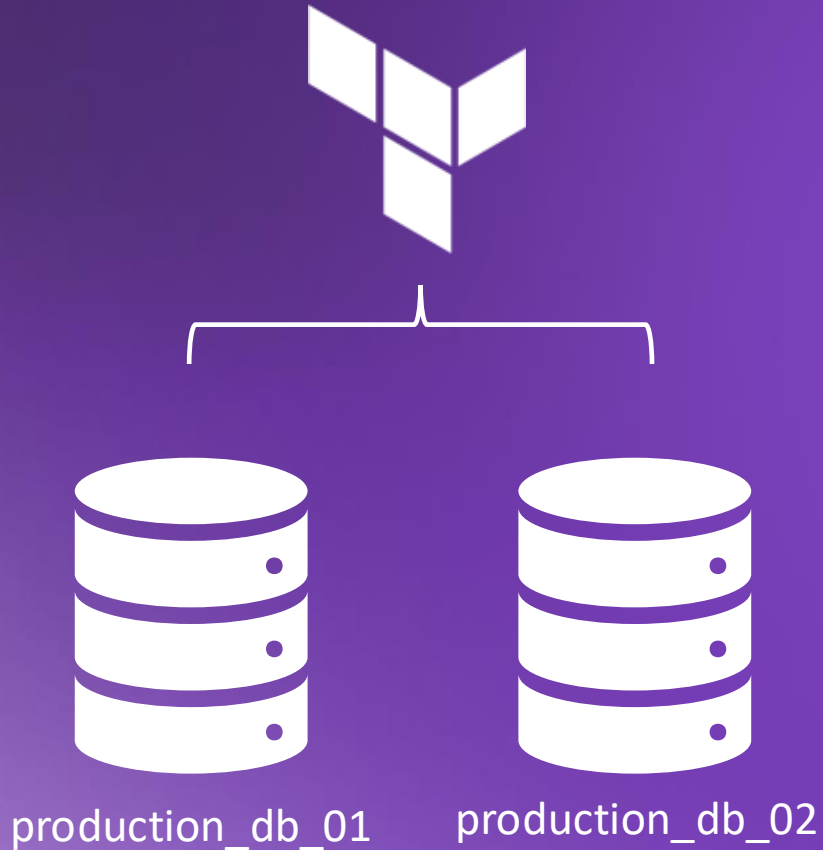
LECTURE

Removed Block





Real-World Scenario



```
database.tf

resource "google_sql_database_instance" "prd_db_1" {
  name                = "production-db-01"
  database_version    = "POSTGRES_15"
  region              = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}

resource "google_sql_database_instance" "prd_db_2" {
  name                = "production-db-02"
  database_version    = "POSTGRES_15"
  region              = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}
```

The database is now critical to production and the DBA team wants manual control over schema changes

What NOT To Do!



```
database.tf

resource "google_sql_database_instance" "prd_db_1" {
  name            = "production-db-01"
  database_version = "POSTGRES_15"
  region          = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}

resource "google_sql_database_instance" "production-db-02" {
  name            = "production-db-02"
  database_version = "POSTGRES_15"
  region          = "us-central1"
  settings {
    tier = "db-f1-micro"
  }
}
```



WAIT! We just want to stop managing it, not DELETE IT!



Removed Block

The `removed` block instructs Terraform to remove a resource from state.

- **The `from` argument**

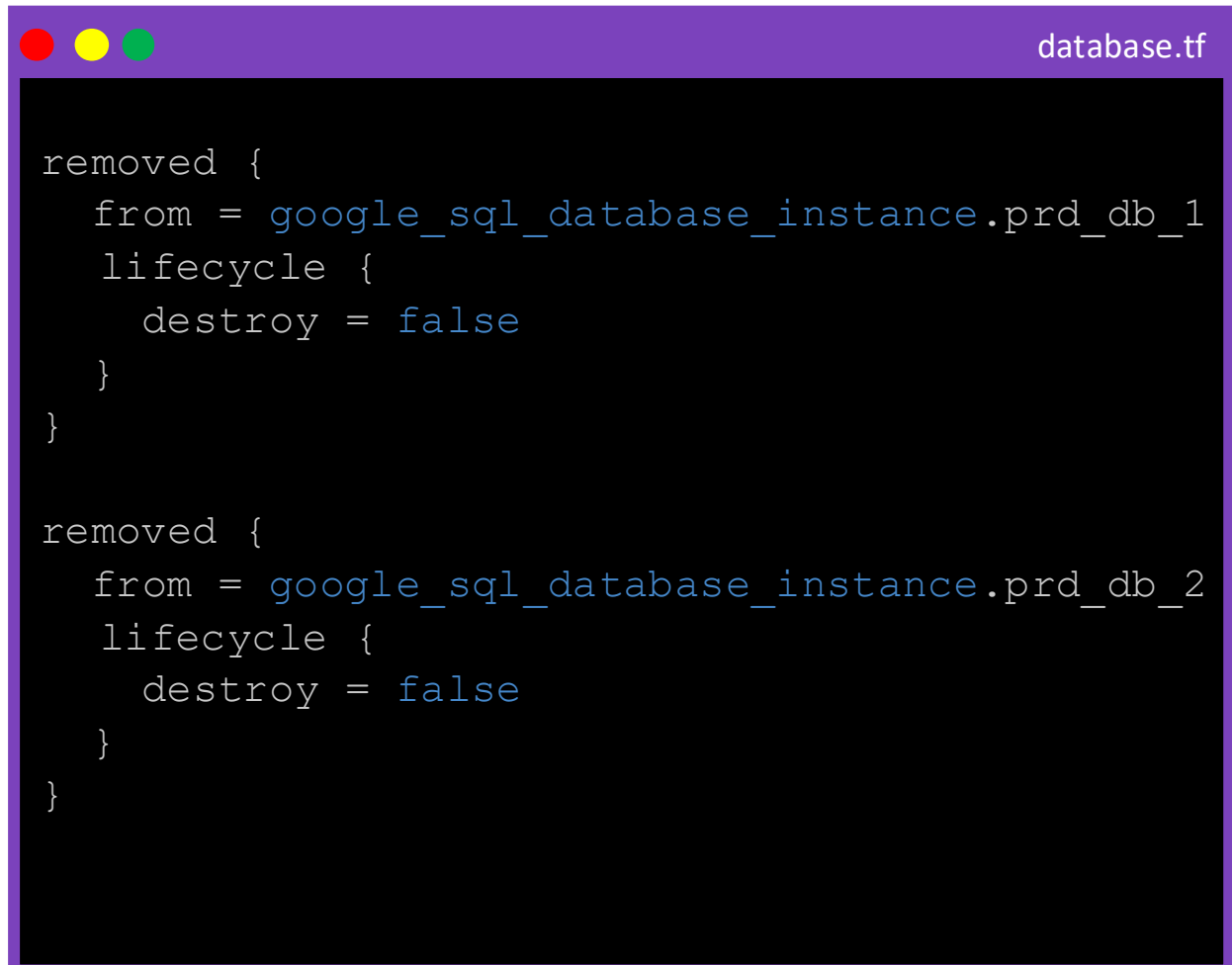
Create standardized infrastructure patterns that can be used across multiple projects and teams.

- **Lifecycle Argument:** `destroy = false`

By default, Terraform removes the resource from state and destroys the actual resource.

- **State Removal Only**

Removes from state file - infrastructure remains untouched



```
removed {
  from = google_sql_database_instance.prddb_1
  lifecycle {
    destroy = false
  }
}

removed {
  from = google_sql_database_instance.prddb_2
  lifecycle {
    destroy = false
  }
}
```

You must also remove the resource block from your config



Key Differences

Compring the Removed Block to the Moved Block

moved Block

vs

removed Block

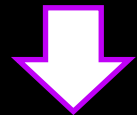
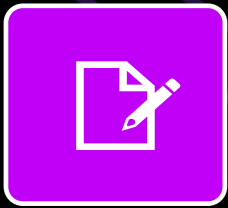
"I'm reorganizing my code, update state to match new addresses"

"I'm done managing this with Terraform, remove it from state"



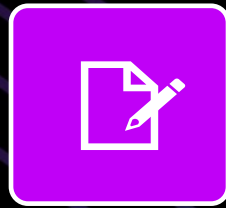
The Refactoring Workflow

Write



Add new removed
block(s) to
Terraform config

Delete



Remove
resource blocks
from config

Plan



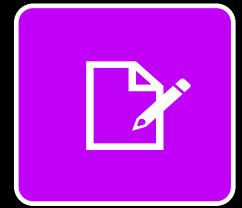
terraform
plan
(validate)

Apply



terraform
apply
(updates state)

Delete



Delete the
removed
blocks

Decision Tree



What Do You Need To Do?

What Block to Use?

- ✧ Renaming resources? → moved block
- ✧ Reorganizing into modules? → moved block
- ✧ Stopping Terraform management? → removed block
- ✧ Handing off to another team? → removed block
- ✧ Splitting configurations? → removed block

LECTURE

Import Block



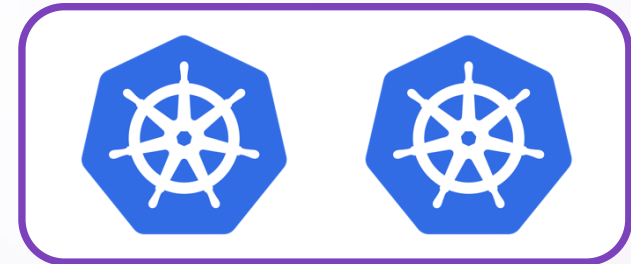
Manage all the Things with Terraform...



```
resource "aws_instance" "web" {  
  ami          = "ami-012345"  
  instance_type = "t2.micro"  
  
  key_name  = "prd-web-key"  
  subnet_id = "subnet_12345abc"  
  
  tags = {  
    Name = "prd-web-svr_01"  
  }  
}  
  
resource "aws_instance" "db" {  
  instance_type = "m5.4xlarge"  
  ami          = "ami-0c55b159"  
}
```

After learning Terraform, you want to start provisioning all resources using Infrastructure as Code

Existing Resources



...but what about these resources already running in the cloud/on-premises?



Import Block

The `import` block in Terraform lets you bring existing infrastructure under Terraform management by mapping real-world resources to your configuration. This allows Terraform to track and manage them without manual setup.

It simplifies migration, reduces effort, and ensures infrastructure is versioned and controlled.





Import Block

Define an input that can be reused and customized to make configurations more flexible and dynamic



Map Existing Resources

Links real-world infrastructure to Terraform without changing it



Simplifies Migration

Reduces manual setup when adopting Terraform



Enables State Management

Bring unmanaged resources under Terraform control



```
# Import Repo
import {
  to = github_repository.prd-app-repo
  id = "my-org/production"
}

# Import Azure Resource Group
import {
  to = azurerm_resource_group.prd-rg
  id = "/subscriptions/1234/rg/my-rg"
}

# Import Vault Secret
import {
  to = vault_kv_secret_v2.my-secret
  id = "secret/data/my-secret"
}
```

Breaking Down the Import Block



```
# Import Repo
```

```
import {
```

```
  to = github_repository.prn-app-repo
```

```
  id = "my-org/production"
```

```
}
```

```
# Import Azure Resource Group
```

```
import {
```

```
  to = azurerm_resource_group.prn-rg
```

```
  id = "/subscriptions/1234/rg/my-rg"
```

```
}
```

```
# Import Vault Secret
```

```
import {
```

```
  to = vault_kv_secret_v2.my-secret
```

```
  id = "secret/data/my-secret"
```

```
}
```

• The import block & curly braces

• What resource block in Terraform will this be mapped to?

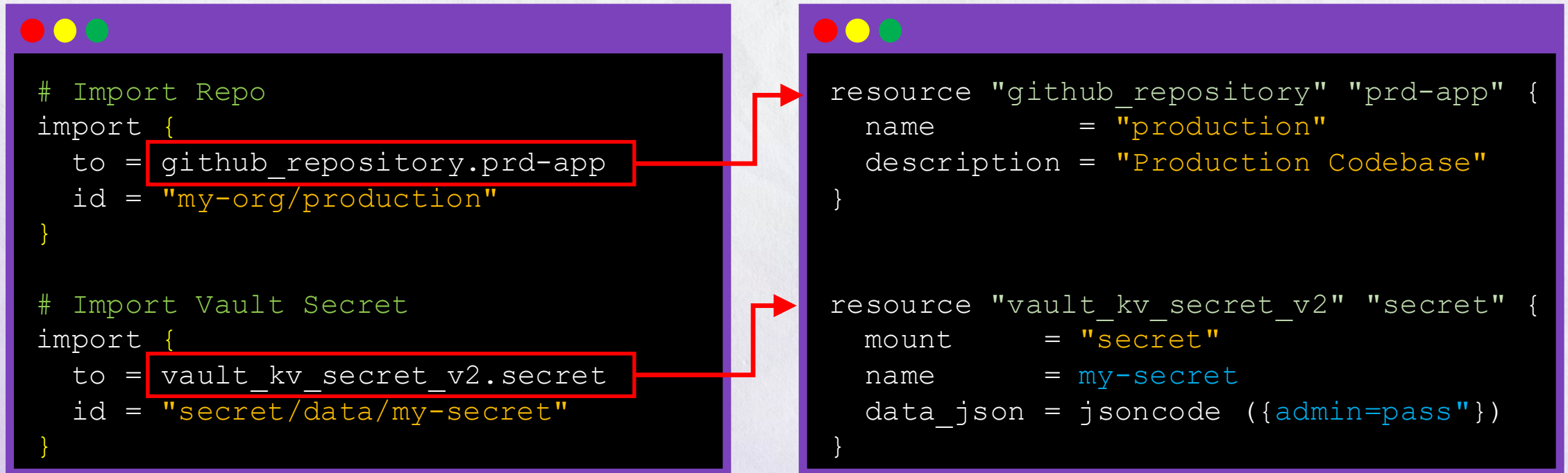
• What is the actual resource that you want to import? A VM? A repo? A network?

• Another example of importing a resource from Vault

Import Block - Requirements



- Before importing a resource, you need to create the resource block that will manage the imported resource moving forward.
- You can write it manually or use `terraform plan -generate-config-out=PATH`



terraform import Command



The original CLI command for importing existing resources into Terraform state

How it works:

- ✈ Run command with resource address and cloud provider ID
- ✈ Resource added to state immediately
- ✈ Still required writing configuration manually
- ✈ No plan step - direct state modification

For the exam: Know that `terraform import` exists, but focus on the `import` block as the modern approach.

You MUST write the resource block first before importing

```
main.tf
resource "aws_instance" "web" {
  ami           = var.prod_ami
  instance_type = m5.medium

  tags = {
    Name = "Production Web"
  }
}
```

```
TERMINAL
$ terraform import aws_instance.web i-12367890abcdef0

aws_instance.web: Importing from ID "i-12367890abcdef0"...
aws_instance.web: Import prepared!
aws_instance.web: Refreshing state...

Import successful!
```